

العرض الأول:

قواعد الكتابة الأساسية:

يعتمد مفسر لغة PHP على وسوم محددة لتعريف بداية ونهاية الكود البرمجي الخاص باللغة، وهما `<?php` و `>?` يقوم المفسر بتنفيذ التعليمات التي تقع بين هذين الوسمين فقط، بينما يتجاهل أي محتوى يقع خارجهما، مما يتيح إمكانية دمج أكواد PHP بمرونة داخل ملفات HTML أو المستندات النصية الأخرى. ومن الناحية البنائية (Syntax) ، يستلزم الأمر وجود فاصل (مثل مسافة فارغة، علامة جدولة، أو سطر جديد) بعد وسم البداية `<?php` مباشرة؛ لضمان تعرف المعالج عليه بشكل صحيح وتجنب حدوث أخطاء في التنفيذ.

الصيغة العامة:

```
1  <?php
2
3  ?>
```

وبالتالي لابد من مراعاة:

1. **حدود التنفيذ:** تتعرف اللغة على الكود المراد تنفيذه من خلال حصره بين علامتين

`<?php`

`?>`

ما يقع خارج هذا النطاق يتم إرساله كما هو دون معالجة.

2. **قواعد الكتابة:** يُشترط وجود فاصل (مسافة أو سطر جديد) بعد وسم البداية `>?php` لتجنب أخطاء البنية (Syntax Errors).

3. **نهاية الجمل البرمجية:** كل جملة برمجية في PHP يجب أن تنتهي بفاصلة منقوطة (؛) نسيانها هو أشهر سبب للأخطاء البرمجية اللغوية.

4. **المسافات البيضاء** :لغة PHP لا تتأثر بكثرة المسافات أو الأسطر الجديدة في الكود خلافاً، فالمفسر يتعامل مع المسافات المتعددة كمسافة واحدة، مما يمنح المتعلم حرية في تنسيق الكود ليكون مقروءاً دون التأثير على عمل البرنامج.

5. **حالة الأحرف للكلمات المحجوزة** :الكلمات الأساسية الخاصة باللغة نفسها مثل (echo, print, if, while, function) لا تتأثر بحالة الأحرف (لا تفرق بين الحروف الكبيرة والصغيرة). فكتابة ECHO هي نفسها echo، وهذا يختلف عن أسماء المتغيرات التي سنتحدث عنها لاحقاً.

التعليقات

نصوص تُكتب داخل الكود البرمجي، يتم تجاهلها من قبل مترجم اللغة (PHP Interpreter) أثناء معالجة الأكواد البرمجية؛ أي أنها لا تُنفذ كأوامر برمجية ولا تظهر نتيقتها للمستخدم النهائي على المتصفح.

فهي عبارة عن أداة تواصل بشري موجهة من المبرمج وإليه (أو لمن يطلع على الكود الذي تمت كتابته من قبل أشخاص آخرون)، وتستخدم للأغراض التالية:

1. التوثيق والشرح :كتابة ملاحظات حول وظيفة جزء معين من الكود لتسهيل فهمه عند العودة إليه مستقبلاً (تُعد بمثابة ذاكرة للمبرمج).

2. التنظيم :تقسيم الكود الطويل إلى أجزاء معنونة ومرتبطة.

3. التنقيح :تعطيل سطر أو مجموعة أسطر مؤقتاً لمنع تنفيذها أثناء تجربة واختبار الكود، دون الحاجة لحذفها نهائياً.

توفر لغة PHP آليتين أساسيتين لإدراج التعليقات داخل الكود البرمجي، وقد صُمم كل نمط منهما ليلتئم غرضاً محدداً من حيث كثافة الشرح وسياق الاستخدام، وهما:

1. تعليقات السطر الواحد (Single-line Comments) :تُستخدم هذه الطريقة لأغراض عدة منها كتابة ملاحظات مختصرة، توضيح دلالة المتغيرات، شرح وظيفة سطر برمجي محدد، بالإضافة إلى منع

أمر معين مؤقتاً عن التنفيذ. فبمجرد كتابة العلامة // أو الرمز # ، يقوم مفسر PHP بتجاهل كل النصوص التي تأتي بعدها حتى نهاية السطر الحالي فقط الذي كتب فيه أحد هذين الرمزتين.

الصيغة العامة:

// هذا تعليق في سطر واحد باستخدام الشرطتين المائلتين

هذا تعليق في سطر واحد باستخدام رمز الشباك

مثال توضيحي:

2. التعليقات متعددة الأسطر (Multi-line Comments) تُستخدم هذه الطريقة لأغراض شمولية منها توثيق الكتل البرمجية، كتابة شروحات تفصيلية للخوارزميات المعقدة، تدوين حقوق الملكية وتاريخ التعديل، فضلاً عن تعطيل مقطع برمجي كامل دفعة واحدة أثناء عملية التنقيح. وتعتمد آليتها على حصر المحتوى بين رمز البداية /* ورمز النهاية */، مما يدفع مفسر PHP لتجاهل وإهمال كافة الأسطر والنصوص الواقعة بين هذين الرمزتين كلياً، بغض النظر عن عددها.

الصيغة العامة:

```
1  /*
2
3  */
```

مثال توضيحي:

```
/* هذا تعليق متعدد الأسطر
يمكنك الكتابة هنا كما تشاء
ولن ينفذ السيرفر أي شيء مما مكتوب هنا
*/
```

نهاية العرض الأول

العرض الثاني

المتغيرات:

تُعرف بأنها أماكن محجوزة في ذاكرة الحاسوب تعمل بمثابة 'أوعية تخزينية' لحفظ البيانات ومعالجتها أثناء عمل البرنامج. ومن الناحية الهيكلية، يُمكن للمتغير أن يحمل اسماً رمزياً مجرداً (مثل \$x, \$y)، أو اسماً دلاليًا وصفيًا (مثل \$age, \$prof_id, \$student_name) يعكس بوضوح طبيعة البيانات المخزنة بداخله؛ لتسهيل قراءة الكود أو تعديله عند الحاجة.

قواعد تسمية المتغيرات في PHP: لتسمية المتغيرات بشكل صحيح، يجب الالتزام بالقواعد التالية:

1. يجب أن يبدأ المتغير بعلامة الدولار (\$)، متبوعة باسم المتغير.

المتغير	الصحة (صح / خطأ)	السبب
\$salary	✓	تم وضع علامة الدولار.
salary	✗	نسيان علامة الدولار يجعل المفسر يبحث عن دالة أو ثابت بهذا الاسم

2. لا يمكن استخدام الرموز الخاصة مثل (\$, #, %, &,) أو المسافات ضمن بنية اسم المتغير؛ ويقتصر فقط استخدام علامة 'الشرطة السفلية' (Underscore _) ؛ حيث تُعتمد كأداة الربط الوحيدة للفصل بين الكلمات في المتغيرات المركبة مثل : (\$student_name)، كما يمكن وضعها في بداية اسم المتغير.

المتغير	الصحة (صح / خطأ)	السبب
\$_name	✓	الشرطة السفلية مسموح أن يبدأ بها اسم المتغير.
*\$name	✗	الرمز * غير مسموح أن يبدأ بها اسم المتغير.

3. لا يمكن أن يبدأ اسم المتغير برقم.

المتغير	الصحة (صح / خطأ)	السبب
---------	------------------	-------

\$student1	✓	الرقم مسموح أن يأتي في منتصف / نهاية اسم المتغير.
\$1student	✗	الرقم غير مسموح أن يأتي في بداية اسم المتغير.

4. أسماء المتغيرات حساسة لحالة الأحرف (Case-sensitive) .

المتغير	الصحة (صح / خطأ)	السبب
\$age	يعتبروا متغيرين وليس واحد	لأن اللغة حساسة لحالة الأحرف
\$Age		

إنشاء المتغيرات:

في لغة PHP ، يتم تعريف المتغير بكتابة علامة متبوعة \$بالاسم، ثم إسناد القيمة له .فلا تمتلك PHP أمراً محدداً للتصريح عن المتغير (Declaration) ، بل يتم إنشاء المتغير تلقائياً في اللحظة التي يتم فيها تعيين قيمة له كما ان القيمة هي التي تحدد نوعه.

الصيغة العامة:

```
القيمة = اسم المتغير $ ;
```

أنواع البيانات للمتغيرات (Variable Data Types) تُصنف لغة PHP ضمن اللغات 'مرنة النوع' وهذا يعني أن المبرمج غير مُلزم بتحديد نوع البيانات (مثل: نصوص، أرقام) عند تعريف المتغير. بدلاً من ذلك، يقوم مفسر اللغة (Interpreter) ببنّي آلية 'استنتاج النوع' (Type Inference) ؛ آلياً بناءً على طبيعة القيمة المسندة إليه في تلك اللحظة.. تدعم اللغة أنواع البيانات التالية:

1. String للنصوص والقيم الحرفية.

2. Int للأعداد الصحيحة (Whole numbers)

3. Float للأعداد العشرية (Decimal numbers)

4. Bool للقيم المنطقية (True / False).
5. Array للمصفوفات (لتخزين قيم متعددة).
6. Object للكائنات البرمجية.
7. Null للمتغيرات الفارغة.
8. Resource للإشارة إلى موارد خارجية (مثل الاتصال بقاعدة بيانات).

• البيانات النصية String:

يتم تحديد نوع البيانات للمتغير كـ "قيمة نصية" في حالة إسناد إما مجموعة من الحروف، الأرقام، الرموز أو خليط منهم داخل علامات تنصيب مفردة أو مزدوجة كقيمة للمتغير؛ ويوجد فارق بين استخدام علامات التنصيب المفردة والمزدوجة فاستخدام علامات التنصيب المفردة (' ') تتيح تخزين الحروف والرموز كما هي مكتوبة بالضبط دون أي معالجة داخلية. أما عند استخدام علامات التنصيب المزدوجة (" ") ، فإن النص يكتسب خاصية "المعالجة النشطة"؛ وهي عملية فحص قبل التخزين للبحث عن أي متغيرات مدمجة، حيث يقوم باستبدال أسمائها البرمجية بقيمها الحقيقية المخزنة في الذاكرة بشكل تلقائي. وفي كلتا الحالتين، يعتمد نجاح بناء الكود (Syntax) على قاعدة "التماثل"، والتي تفرض غلق النص بنفس نوع العلامة التي بدأ بها، لضمان استقرار القيمة داخل الذاكرة وظهور المخرجات بشكل صحيح عند الاستدعاء.

الصيغة العامة لتحديد نوع البيانات النصي:

```
قيمة المتغير = اسم المتغير $  
القيمة توضع بين علامة تنصيب ' ' أو علامتي تنصيب " "
```

مثال توضيحي:

```
$first_name = "nermeen";  
$last_name = 'ahmed';
```

• الأرقام الصحيحة (Integer)

يتم تحديد نوع البيانات كرقم صحيح من خلال إسناد قيمة للمتغير عبارة عن رقم صحيح لا يحتوي على فواصل عشرية أو كسور، مثل أعمار الطلاب أو عدد المواد الدراسية. وبه يتم كتابة الرقم مباشرة دون الحاجة لوضعه داخل علامات تنصيص وإلا يتم اعتباره نصاً؛ لكي يتمكن المفسر من التعرف عليه كقيمة حسابية يمكن استخدامه داخل العمليات الحسابية. وتستوعب هذه البيانات نطاقاً واسعاً من الأرقام، سواء كانت موجبة أو سالبة أو صفراً، بشرط أن تظل ضمن الحدود التي تسمح بها ذاكرة الحاسوب (والتي تصل إلى أكثر من ملياري رقم في الاتجاهين الموجب والسالب). ويعد اختيار هذا النوع من البيانات أمراً ضرورياً لضمان سرعة تنفيذ العمليات الرياضية داخل البرنامج التعليمي وتجنب استهلاك مساحة الذاكرة في تفاصيل غير ضرورية.

الصيغة العامة لتحديد نوع البيانات العددي الصحيح:

قيمة المتغير = اسم المتغير \$
القيمة عددية صحيحة //

🚩 مثال توضيحي:

```
$number1 = 1;  
$number2 = 236985214;  
$number3 = -298;
```

ملاحظات الأعداد الصحيحة:

- يجب أن يحتوي العدد الصحيح على خانة (رقم) واحدة على الأقل.
- يجب ألا يحتوي العدد الصحيح على فاصلة عشرية.
- يمكن أن يكون العدد الصحيح موجباً أو سالباً.

- يمكن تحديد الأعداد الصحيحة بعدة أنظمة: العشري (أساس 10)، الست عشري (أساس 16)، الثماني (أساس 8)، أو الثنائي (أساس 2).

• الأرقام العشرية (Float / Double) :

يتم تحديد نوع البيانات ك رقم عشري من خلال إسناد قيمة للمتغير تحتوي على فواصل أو كسور عشرية، مثل درجات الطلاب (كمثال: 18.5) أو النسب المئوية للنجاح. ويكتب هذا الرقم مباشرة داخل الكود دون علامات تنصيص؛ وذلك لضمان تمييزه كقيمة رياضية دقيقة وليس مجرد نص، مما يسمح للبرنامج بإجراء العمليات الحسابية التي تتطلب دقة عالية. وتستوعب هذه البيانات نطاقاً واسعاً جداً من الأرقام، سواء كانت قيماً صغيرة جداً بكسور دقيقة أو أرقاماً ضخمة تُكتب بالصيغة الأسية. ويُعد اختيار هذا النوع من البيانات أمراً حيوياً في بناء البرامج التعليمية عند التعامل مع نتائج الاختبارات أو القياسات الحسابية التي لا تقبل التقريب، مع مراعاة أن لغة PHP تخصص لها مساحة في الذاكرة تضمن الحفاظ على موضع الفاصلة العشرية ودقتها مهما تغيرت القيمة الرقمية.

الصيغة العامة لتحديد نوع البيانات العددي العشري:

قيمة المتغير = اسم المتغير \$
القيمة عددية عشرية //

```
$number1 = 1.5;  
$number2 = 5257.35;
```

🌈 مثال توضيحي:

• القيم المنطقية (Boolean)

يتم تحديد نوع البيانات ك قيمة منطقية فور إسناد إحدى القيمتين للمتغير: إما الصواب (True) أو الخطأ (False)، وهي تُشبه إلى حد كبير مفاتيح التشغيل (ON/OFF) وتُكتب هذه القيم مباشرة في الكود دون علامات تنصيص؛ لكي يميزها البرنامج كقيم منطقية محددة وليست مجرد نصوص عادية. وتُعد هذه البيانات هي حجر الزاوية في بناء "المنطق البرمجي" داخل التطبيقات التعليمية؛ فهي المسؤولة عن اتخاذ

القرارات، مثل تحديد ما إذا كانت إجابة الطالب "صحيحة" أم "خاطئة"، أو ما إذا كان الطالب "مؤهلاً" للانتقال للمستوى التالي أم لا. ومن الناحية الأكاديمية، تخصص لغة PHP لهذا النوع أصغر مساحة ممكنة في الذاكرة، حيث لا يحتاج المتغير هنا لتخزين أرقام أو حروف، بل يحتاج فقط لتخزين "حالة المنطق" التي يعتمد عليها البرنامج في توجيه مساره.

الصيغة العامة لتحديد نوع البيانات المنطقي:

```
; قيمة المتغير = اسم المتغير $  
القيمة منطقية //
```

🚦 مثال توضيحي :

```
$status1 = true;  
$status0 = false;
```

2. الأنواع المركبة (Compound Types)

تنتقل لغة PHP إلى مستوى أكثر تقدماً من تنظيم البيانات من خلال الأنواع المركبة، وهي أوعية برمجية فائقة القدرة لا تكتفي بحمل قيمة واحدة فقط، بل صُممت لتكون "حاويات ذكية" قادرة على استيعاب مجموعة متعددة ومتنوعة من البيانات داخل متغير واحد. فبينما تحمل الأنواع البسيطة (مثل الأرقام) قيمة مفردة، تتيح الأنواع المركبة للمبرمج بناء هياكل بيانات معقدة ومنظمة.

• المصفوفات (Arrays)

يتم تحديد نوع البيانات ك مصفوفة عند الحاجة لجمع قائمة من القيم المتعددة التي لها نفس النوع وتخزينها داخل متغير واحد بدلاً من حجز متغير مستقل لكل قيمة، وهي تُشبه إلى حد كبير "مخزن الملفات" التي تحتوي على عدة أجزاء، كل جزء يحمل عناصر مختلفة لكنهم جميعاً ينتمون لنفس المخزن. وتسمح المصفوفات بتخزين أنواع مختلفة من البيانات معاً (نصوص، أرقام، أو حتى قيم منطقية) في هيكل واحد منظم يسهل الوصول إليه عبر ما يسمى بـ "المفتاح" أو "الفهرس". وتُعد المصفوفات العنصر الأساسي لتنظيم البيانات الضخمة في التطبيقات التعليمية؛ فهي المسؤولة عن تخزين قوائم أسماء الطلاب، أو مجموعة أسئلة الاختبار،

أو درجات مواد دراسية كاملة في مكان واحد. كما تكمن قوة المصفوفات في قدرتها على إدارة مساحة الذاكرة بشكل منظم، حيث تتيح للمبرمج معالجة مئات القيم دفعة واحدة باستخدام أوامر برمجية بسيطة، مما يقلل من تكرار الكود ويزيد من سرعة استجابة البرنامج التعليمي.

الصيغة العامة لكتابة نوع البيانات مصفوفة:

```
اسم الدالة$ = array (قيمة1 ، قيمة2 ، قيمة3، ....);
```

مثال توضيحي:

```
$students_grad=array("excellent", "very_good","good",  
,"acceptable" , "weak" , "failed") ;
```

• الكائنات (Objects)

يتم تحديد نوع البيانات ككائن عندما نريد تمثيل "كيان متكامل" يجمع بين البيانات والأفعال الخاصة به في وحدة واحدة، وهو المفهوم الأساسي فيما يعرف بـ البرمجة كائنية التوجه (OOP) ويمكن تشبيه الكائن بـ "المنتج النهائي" الذي يتم تصنيعه بناءً على "مخطط هندسي" يُسمى الفئة أو (Class) ؛ فإذا كانت الفئة هي مخطط لبناء "طالب"، فإن الكائن هو "الطالب محمد" الذي يمتلك بيانات خاصة (اسمه وعمره) ولديه قدرات أو وظائف (مثل تسجيل الدخول أو حل الاختبار). وتتميز الكائنات عن المصفوفات بأنها لا تخزن مجرد قوائم بيانات، بل تخزن معها "الدوال" التي تحدد سلوك هذا الكائن وكيفية تفاعله داخل النظام التعليمي. كما تمنح الكائنات البرنامج مرونة فائقة وقدرة على إعادة استخدام الكود (Reuse) ، حيث تتيح بناء أنظمة معقدة ومنظمة تشبه الواقع تماماً، مما يسهل عملية التطوير والصيانة ويجعل الكود أكثر تنظيماً وقابلية للتوسع.

الصيغة العامة لكتابة نوع البيانات كائن: (الخصائص) اسم الكلاس new = اسم الكائن\$

مثال توضيحي:

```
class student_info
{
    public $name;
    public $id;
}

$student1 = new student_info("nermeen", 2);
```

3. الأنواع الخاصة (Special Types)

تتعامل لغة PHP مع حالات استثنائية في البرمجة من خلال الأنواع الخاصة، وهي أدوات برمجية فريدة لا تُستخدم لتخزين البيانات العادية، بل لوصف "حالة" المتغير أو "مصدر" خارجي للبيانات. ومن أبرز هذه الأنواع:

- القيمة الفارغة (NULL)

يتم تحديد نوع البيانات كقيمة فارغة (NULL) عندما يكون المتغير موجوداً كاسم ومكان محجوز، ولكنه لا يحتوي على أي قيمة أو نوع بيانات على الإطلاق. ويمكن تشبيهه (NULL) بـ "صندوق بريد فارغ"؛ الصندوق له عنوان ومعروف مكانه (اسم المتغير)، ولكن لا توجد بداخله رسالة أو طرد (القيمة). وتختلف هذه الحالة تماماً عن الصفر أو النص الفارغ، فهي تعني "غياب البيانات" عمداً. وتُعد (NULL) أداة ذكية في البرامج التعليمية لتحديد الحالات التي لم تبدأ بعد؛ مثل متغير "درجة الطالب" قبل أن يؤدي الاختبار، حيث تُسند له القيمة (NULL) لنخبر البرنامج أن هذا الطالب لم يحصل على درجة بعد (وليس درجته صفراً). كما تساعد القيمة الفارغة في توفير مساحة الذاكرة ومنع حدوث أخطاء برمجية عند استدعاء متغيرات لم يتم ملؤها، مما يمنح المبرمج تحكماً دقيقاً في منطق البرنامج وضمان استقرار النظام التعليمي عند التعامل مع البيانات المفقودة أو الاختيارية.

الصيغة العامة لكتابة نوع البيانات الفارغ:

```
$اسم المتغير = null;
```

مثال توضيحي:

```
$var_null = null;
```

• الموارد (Resources)

يتم تحديد نوع البيانات ك مورد (Resource) عندما لا يحمل المتغير قيمة نصية أو رقمية عادية، بل يعمل ك "رابط ذكي" أو إشارة لفتح قناة اتصال مع مصدر خارجي خارج بيئة لغة PHP. ويمكن تشبيه المورد بـ "جهاز التحكم عن بُعد (Remote Control)؛" فالجهاز في حد ذاته ليس هو التلفاز، ولكنه الوسيلة الوحيدة والمباشرة للتحكم فيه وإدارته. وتُستخدم الموارد في البرامج التعليمية عند الحاجة للتعامل مع ملفات خارجية) مثل فتح وقراءة ملف PDF للمنهج(، أو عند الاتصال بقاعدة بيانات الطلاب (MySQL) لجلب بياناتهم. ومن الناحية الأكاديمية، لا تُعد الموارد نوعاً من البيانات التي يمكن تعديلها مباشرة، بل هي "معارف خاصة" تُنشأ بواسطة دوال محددة لتسمح للمبرمج بالسيطرة على موارد النظام الخارجية؛ مما يضمن كفاءة عالية في تبادل البيانات بين لغة البرمجة والمصادر الأخرى دون استهلاك لذاكرة الحاسوب، ويحافظ على استقرار وتأمين الاتصال بالمحتوى التعليمي الخارجي.

الصيغة العامة لكتابة اسم المورد:

```
$var_resource = mysql_connect("lochabast,.user, password).database());
```

مثال توضيحي:

؛ () دالة الاتصال = اسم المتغير \$

نهاية العرض الثاني

أوامر الطباعة (Output Commands)

الأوامر البرمجية المسؤولة عن عرض النتائج النهائية ومخرجات الكود على متصفح الويب؛ فهي تعمل بمثابة "المُترجم التقني" الذي يتولى استدعاء السلاسل النصية (Strings) والبيانات الرقمية ومعلومات الوسائط المتعددة من ذاكرة الخادم، ثم معالجتها وتمريرها لتظهر كمخرجات مرئية في واجهة المتصفح:

• الأمر echo

أداة أساسية لإخراج وعرض النتائج من الخادم إلى متصفح المستخدم، ويتميز بالسرعة في التنفيذ والمرونة في الكتابة، قد يتم استخدامه بأقواس () أو بدون كما في المثال بالصورة.

الصيغة العامة لكتابة الأمر echo:

; نكتب هنا ما نريد طباعته بأقواس أو بدون أقواس echo

🚩 مثال توضيحي لطباعة نص على الشاشة:

```
; "مرحبا بك" echo
// يمكن وضع النص أو المتغير الذي نريد طباعته بين قوسين
echo("مرحبا بك");
```

كما يدعم وسوم لغة الترميز (HTML): حيث يتميز بالقدرة على معالجة وتضمين وسوم لغة HTML ضمن سطر الطباعة؛ مما يتيح للمطور التحكم في تنسيق المخرجات التعليمية وظهرها بشكل منظم على واجهة المتصفح.

🚩 مثال توضيحي لطباعة وسوم لغة html:

```
echo "<h2> تعلم مهارات لغة php </h2>";  
echo "<br>المهارات الأساسية";
```

ويمكن إسناد القيمة للمتغير وطباعتها: يتيح الأمر echo إمكانية استدعاء القيم المخزنة داخل المتغيرات البرمجية وعرضها للمتعلم بشكل ديناميكي؛ حيث يمكن إسناد قيمة معينة (سواء كانت نصية، عددية، أو نتيجة عملية حسابية) إلى متغير، ثم استخدام الأمر لطباعة هذه القيمة مباشرة. وتظهر أهمية هذه الخاصية في بيانات التعلم الرقمية عند عرض بيانات متغيرة مثل (اسم الطالب، أو درجات الاختبارات).

🚩 مثال توضيحي لطباعة قيمة متغير:

ملاحظات: تُحاط القيم النصية (Strings) بعلامات تنصيص عند الطباعة، ولكن هناك فرق بين العلامات المفردة والمزدوجة في لغة PHP :

```
$txt1 = " تعلم لغة php ";  
$txt2 = "ضمن دورة برمجة قواعد بيانات الويب";  
  
echo "<h2>$txt1</h2>";  
echo "<p> $txt2</p>";
```

أولاً: علامة التنصيص المزدوجة "

تتميز هذه العلامات بقدرتها على "تحليل" المحتوى؛ حيث تسمح بإدراج المتغيرات مباشرة داخل النص، ويقوم الخادم تلقائياً باستبدال اسم المتغير بقيمته الفعلية عند العرض

✚ مثال توضيحي لاستخدام علامة التنصيص المزدوجة:

```
// إسناد قيمة للمتغير (اسم الطالب كمثال)
$student_name = "نرمين ونيس";

// استخدام علامات التنصيص المزدوجة لإدراج المتغير مباشرة
echo "في منصتنا التعليمية $student_name أهلاً بك يا";
```

ثانياً: علامة التنصيص المفردة '

عند وضع المتغير مباشرة داخل علامات التنصيص المفردة، فإن لغة PHP لا تقوم بمعالجته أو استخراج قيمته، بل تتعامل معه كأنه "نص عادي" أو مجرد حروف مكتوبة. وهذا يؤدي إلى ظهور اسم المتغير نفسه (بعلامة \$) للمستخدم بدلاً من القيمة المخزنة داخله.

✚ مثال توضيحي علامة التنصيص المفردة:

```
// هنا سيتم طباعة اسم المتغير كما هو وليس القيمة
echo '$score: درجتك في الاختبار هي';
```

وبالتالي يجب استخدام معامل الربط وهو "النقطة (.)"؛ حيث يعمل هذا المعامل كحلقة ربط تقوم بدمج السلسلة النصية الثابتة مع القيمة الديناميكية للمتغير، مما يضمن عرض النتائج للمتعلم بشكل صحيح دون تداخل مع النص البرمجي كالاتي.

✚ مثال توضيحي لاستخدام معامل الربط:

```
// إسناد قيمة للمتغير
$course_name = "برمجة الويب";

// استخدام علامات التنصيص المفردة مع معامل الربط (النقطة)
echo 'ضمن المقررات ' . $course_name . ' مرحباً بكم في مقرر';
```

الأمر print:

يُستخدم كأداة إخراج تتيح عرض النتائج والبيانات على المتصفح، ويُعامل برمجياً كدالة بسيطة تمنح المبرمج قيمة مسترجعة (تساوي 1) للتأكد من نجاح عملية الطباعة، مما يجعله مناسباً للاستخدام داخل التعبيرات المنطقية، قد يتم استخدامه بأقواس أو بدون.

الصيغة العامة لكتابة الأمر echo:

```
print
```

 ; نكتب هنا ما نريد طباعته بأقواس أو بدون أقواس

مثال توضيحي لطباعة نص على الشاشة:

دعم وسوم لغة الترميز (HTML): حيث يتميز بالقدرة على معالجة وتضمين وسوم لغة HTML ضمن سطر الطباعة؛ مما يتيح للمطور التحكم في تنسيق المخرجات التعليمية وظهورها بشكل منظم على واجهة المتصفح.

```
print " مرحبا بك " ;
```

```
// ( ) يمكن وضع النص أو المتغير الذي نريد طباعته بين قوسين
```

```
print(" مرحبا بك ");
```

مثال توضيحي لطباعة وسوم لغة html:

```
print " <h2> تعلم مهارات لغة php </h2>";
```

```
print "<br>المهارات الأساسية";
```

إسناد القيمة للمتغير وطباعتها: يتيح الأمر print إمكانية استدعاء القيم المخزنة داخل المتغيرات البرمجية وعرضها للمتعلم بشكل ديناميكي؛ حيث يمكن إسناد قيمة معينة (سواء كانت نصية، عددية، أو نتيجة عملية حسابية) إلى متغير، ثم استخدام الأمر لطباعة هذه القيمة مباشرة. وتتجلى أهمية هذه الخاصية في بيئات التعلم الرقمية عند عرض بيانات متغيرة مثل (اسم الطالب، أو درجات الاختبارات)

مثال توضيحي لطباعة قيمة متغير:

```
$txt1 = " تعلم لغة php ";
$txt2 = "ضمن دورة برمجة قواعد بيانات الويب";

print "<h2>$txt1</h2>";
print "<p> $txt2 </p>";
```

ملاحظات: تُحاط القيم النصية (Strings) بعلامات تنصيص عند الطباعة، ولكن هناك فرق بين العلامات المفردة والمزدوجة في لغة PHP :

أولاً: علامة التنصيص المزدوجة " تتميز هذه العلامات بقدرتها على "تحليل" المحتوى؛ حيث تسمح بإدراج المتغيرات مباشرة داخل النص، ويقوم الخادم تلقائياً باستبدال اسم المتغير بقيمته الفعلية عند العرض.

✚ مثال توضيحي لاستخدام علامة التنصيص المزدوجة:

```
// إسناد قيمة للمتغير (اسم الطالب كمتال)
$student_name = "نرمين ونيس";

// استخدام علامات التنصيص المزدوجة لإدراج المتغير مباشرة
print "في منصتنا التعليمية $student_name أهلاً بك يا";
```

ثانياً: علامة التنصيص المفردة ' '

عند وضع المتغير مباشرة داخل علامات التنصيص المفردة، فإن لغة PHP لا تقوم بمعالجته أو استخراج قيمته، بل تتعامل معه كأنه "نص عادي" أو مجرد حروف مكتوبة. وهذا يؤدي إلى ظهور اسم المتغير نفسه (بعلامة \$) للمستخدم بدلاً من القيمة المخزنة داخله.

✚ مثال توضيحي لاستخدام علامة التنصيص المفردة:

```
$score = 95;

// هنا سيتم طباعة اسم المتغير كما هو وليس القيمة
print '$score: درجتك في الاختبار هي';
```

وبالتالي يتم استخدام معامل الربط وهو "النقطة (.)" ؛ حيث يعمل هذا المعامل كحلقة ربط تقوم بدمج السلسلة النصية الثابتة مع القيمة الديناميكية للمتغير، مما يضمن عرض النتائج للمتعلم بشكل صحيح دون تداخل مع النص البرمجي كالاتي.

🚩 مثال توضيحي لاستخدام معامل الربط:

```
$course_name = "برمجة الويب";  
// استخدام النقطة لربط النص بالمتغير  
print 'مرحباً بكم في مقرر ' . $course_name . ' ضمن المقررات';
```

الدالة (var_dump):

دالة برمجية تحليلية تُستخدم لاستخراج البيانات الوصفية (Metadata) الخاصة بالمتغيرات، حيث تتميز بقدرتها على تقديم تقرير هيكلي شامل يتضمن ثلاثة مستويات من المعلومات وهي نوع البيانات (Data Type)، والقيمة الفعلية (Value)، والحجم أو الطول (Length/Size) فعلى عكس أوامر الإخراج السابقة التي تكتفي بعرض القيم النصية، تعمل (var_dump()) على تحليل تفاصيل عن المتغير برمجياً لتحديد نوعه مثل الأعداد الصحيحة (int) أو القيم المنطقية (bool) أو المصفوفات (array) ؛ مما يبرز أهميتها في كشف القيم "غير المرئية" في المتصفح مثل القيمة الفارغة (NULL) والقيم المنطقية الزائفة (false) التي قد لا تظهر عند استخدام أوامر الطباعة العادية. وفي هذا السياق تُمثل الدالة أداة محورية في مرحلة "تتبع الأخطاء" (Debugging) تتيح للمتعلم فهم كيفية معالجة المترجم (Compiler) للبيانات في الذاكرة، ويساهم هذا التحليل الدقيق في ضمان استقرار الأنظمة التعليمية من خلال التحقق من مطابقة أنواع البيانات المسترجعة من قواعد البيانات مع المتطلبات المنطقية للكود قبل التنفيذ، مما يقلل من احتمالية حدوث أخطاء التعارض البرمجي.

```
$name = 'nermeen';
```

```
var_dump($name);
```

```
$x = 5985;  
var_dump($x);
```

```
$x = 10.365;  
var_dump($x);
```

```
$x = true;  
var_dump($x);
```

```
string(7) "nermeen"
```

```
int(5985)
```

```
float(10.365)
```

```
bool(true)
```

نهاية العرض الثالث

العرض الرابع

تحويل أنواع البيانات (Type Casting) :

إجراءً برمجياً يُستخدم لإعادة تعريف المتغير من نوع إلى آخر بشكل مباشر، حيث تتميز هذه العملية بمنح الفرد تحكماً كلياً ودقيقاً في طبيعة القيم المخزنة داخل الذاكرة. مثل تحويل الأعداد العشرية (Float) إلى أعداد صحيحة (Integer) أو تحويل القيم النصية إلى منطقية؛ مما يضمن توافقها التام مع العمليات الحسابية المستهدفة.

كما تمثل أداة محورية لضبط مدخلات النظام وتجهيزها للمعالجة، حيث تتيح للمتعلم صياغة كود أكثر دقة وموثوقية من خلال فرض النوع المطلوب على المتغير قبل التنفيذ. ويساهم هذا التحكم المباشر في تعزيز استقرار الأنظمة التعليمية عبر منع حدوث أخطاء "عدم تطابق الأنواع" (Type Mismatch)، ويضمن معالجة البيانات المسترجعة من المستخدم أو قواعد البيانات بالشكل المتوافق مع بنية البرنامج، مما يقلل من احتمالية حدوث أخطاء منطقية أو مخرجات غير دقيقة داخل النظام.

يتم تنفيذ عملية التحويل من خلال وضع "نوع التحويل" المطلوب محاطاً بأقواس (مباشرة قبل المتغير أو القيمة المراد تغيير نوعها. وتعمل هذه الأقواس بمثابة "مُعامل تحويل" يُجبر محرك لغة PHP على إعادة تفسير القيمة المخزنة وفقاً للنوع المذكور بين القوسين (مثل تحويل متغير نصي إلى عددي)، مما يمنح المطور سيطرة كاملة على كيفية معالجة البيانات داخل الذاكرة ويمنع التضارب البرمجي الناتج عن التحويلات التلقائية غير المتوقعة.

؛ اسم المتغير المراد تغيير نوعه \$ (نوع البيانات) = اسم المتغير \$

معاملات التحويل:

- المعامل (string): يُستخدم لتحويل القيمة إلى نوع بيانات نصي، وهو ضروري عند الحاجة لدمج الأرقام أو القيم المنطقية داخل جمل نصية.
- المعامل (int): يعمل على تحويل القيمة إلى عدد صحيح، حيث يقوم بتجاهل الكسور العشرية أو تحويل النصوص الرقمية إلى قيم حسابية.
- المعامل (float): يُستخدم لتحويل القيمة إلى عدد عشري، مما يسمح بإجراء العمليات الحسابية التي تتطلب دقة عالية في الكسور.
- المعامل (bool): يقوم بتحويل القيمة إلى نوع منطقي (true) أو (false)، ويُستخدم بكثرة في فحص حالات التحقق والشرط البرمجي.
- المعامل (array): يُستخدم لتحويل القيم المنفردة أو الكائنات إلى مصفوفة، مما يسهل عملية تخزين وتنظيم البيانات في مجموعات.
- المعامل (object): يعمل على تحويل القيمة إلى كائن (Object)، وهو ما يخدم توجه البرمجة كائنية المنحى (OOP) داخل النظام.
- المعامل (unset): وهو معامل يُصنف حالياً كـ "ملغى أو مهمل (Deprecated)"، وكان يُستخدم سابقاً لتحويل القيمة إلى نوع فارغ (NULL).

مثال توضيحي لكيفية تحويل أنواع البيانات:

```
$a = 5;           // Integer
$b = 5.34;        // Float
$c = "hello";     // String
$d = true;        // Boolean
$g = NULL;        // NULL
// اختيار نوع المتغير كالآتي
$a = (int) $a;
$b = (int) $b;
$c = (int) $c;
$d = (int) $d;
$g = (int) $g;

// استخدام الدالة هنا var_dump() لمعرفة نوع المتغير بعد تغيير نوعه
var_dump($a);
var_dump($b);
var_dump($c);
var_dump($d);
var_dump($g);
```

```
int(5)
int(5)
int(0)
int(0)
int(0)
```

الثوابت (Constants) :

أوعية تخزينية تُستخدم لحفظ القيم التي تظل ثابتة طوال فترة تنفيذ البرنامج، حيث تتميز بكونها غير قابلة للتعديل أو الحذف بمجرد إتمام عملية تعريفها. فعلى عكس المتغيرات التي تتسم بمرونة تغيير محتواها، تعمل الثوابت على تثبيت القيمة الرقمية أو النصية لقيمة معينة في الذاكرة؛ مما يضمن حماية البيانات الحساسة (مثل إعدادات قاعدة البيانات أو المسارات الأساسية للنظام) من التغيير العرضي أو التداخل البرمجي أثناء التشغيل.

وتمثل الثوابت أداة محورية لتعزيز أمن وموثوقية الأكواد البرمجية، حيث تتيح للمتعم بناء هيكل برمجي متماسك يعتمد على "مرجعيات ثابتة" لا تتأثر بالعمليات المنطقية المتغيرة داخل النظام. ويساهم هذا الالتزام بثبات القيم في تعزيز استقرار الأنظمة التعليمية عبر منع حدوث أخطاء "إعادة التعيين غير المقصودة"، ويضمن أن

الثوابت الجوهرية التي يعتمد عليها منطق البرنامج تظل مطابقة لمواصفاتها الأصلية منذ لحظة البدء وحتى انتهاء التنفيذ، مما يقلل من احتمالية حدوث أخطاء في البيانات أو انهيار في بنية النظام التعليمي.

الصيغة العامة لكتابة اسم الثابت:

تُحدد لغة PHP مجموعة من قواعد التسمية للثوابت، والتي تُميزها هيكلياً ووظيفياً عن المتغيرات العادية، بدء اسم الثابت بحرف هجائي أو شرطة سفلية (_) فقط مع منع استخدام علامة الدولار (\$) التي تُميز المتغيرات. ويُعد الالتزام بهذه القواعد أداة محورية لضبط جودة الكود البرمجي، حيث تتيح للمتعلم التمييز البصري الفوري بين القيم الثابتة والقيم المتغيرة داخل النظام، مما يعزز من مقروئية البرامج التعليمية واستقرارها التقني.

الدالة define():

الأداة البرمجية الأساسية لتعريف الثوابت، حيث تُستخدم لإنشاء قيم مرجعية مستقرة لا يمكن تغييرها أو إعادة تعريفها بمجرد إنشائها أثناء وقت التشغيل، وتتميز هذه الدالة بتعريف ثوابت على نطاق عام مما يجعلها متاحة للاستخدام في كافة أجزاء الكود دون الحاجة لإجراءات استدعاء معقدة؛ وهو ما يضمن استقرار البيانات للنظام. ومع ذلك يفرض المنطق البرمجي قيوداً تقنياً يمنع استخدامها لتعريف ثوابت داخل الفئات (Classes) ، حيث يقتصر دورها على النطاق العام للملف؛ مما يتطلب التمييز بينها وبين طرق التعريف الأخرى عند بناء الأنظمة القائمة على البرمجة كائنية التوجه لضمان التوازن بين شمولية البيانات العامة وخصوصية المكونات البرمجية.

الصيغة العامة لتعريف ثابت:

```
define( "اسم الثابت", قيمة الثابت );
```

قيمة الثابت: إذا كانت نصية توضع بين علامتي تنصيص.

مثال توضيحي:

```
define("name", "nermeen wanees");
```

الأمر const:

الكلمة المفتاحية المخصصة لتعريف الثوابت خلال مرحلة التجميع وهي تختلف وظيفياً عن الدالة `define()` في كونها تركز على فلسفة البرمجة كائنية التوجه (OOP) وتكمن القيمة العلمية لهذه الأداة في قدرتها على تعريف الثوابت داخل الفئات (Classes) ، مما يحقق مبدأ التغليف (Encapsulation) ويضمن خصوصية البيانات المرتبطة بكل مكون برمجى على حدة؛ وهو ما يمنع تداخل الثوابت العامة مع الخصائص الفرعية للمكونات التعليمية الرقمية، كما تساهم `const` في تحسين كفاءة أداء النظام؛ حيث يتم تخصيص موارد الذاكرة لها قبل بدء التنفيذ الفعلي للكود، مما يقلل من العمليات الحسابية أثناء وقت التشغيل. وتفرض هذه الأداة نظاماً هيكلياً صارماً يمنع تعريف الثوابت داخل الأطر الديناميكية كالجمل الشرطية أو الدوال، وهو ما يعد ميزة تنظيمية تضمن استقرار البناء الهيكلي للتطبيق.

الصيغة العامة لتعريف ثابت:

```
قيمة الثابت = اسم الثابت const
```

قيمة الثابت: إذا كانت نصية توضع بين علامتي تنصيص.

مثال توضيحي:

```
const age = 25 ;
```

مهارات التحكم في المتغيرات وفحص البيانات

تُشكل مهارات التحكم في المتغيرات وفحص البيانات الركيزة الأساسية لضمان استقرار تطبيقات الويب وتأمين تدفق البيانات داخل نظام إدارة قواعد البيانات. ونظراً لأن مدخلات المستخدمين عبر المتصفح تكون ديناميكية وغير متوقعة في كثير من الأحيان، فإن استخدام دوال الفحص والتحويل يُمثّل خط الدفاع الأول للمطور؛ حيث تضمن هذه الأدوات التحقق من سلامة البيانات، وإدارة موارد الخادم بكفاءة، وتجنب الأخطاء البرمجية الناتجة عن استدعاء متغيرات غير معرفة أو التعامل مع أنماط بيانات غير متوافقة.

دالة تحويل نوع المتغير (settype)

الدالة المسؤولة عن إعادة تحديد نوع البيانات للمتغيرات. وتبرز وظيفتها في تعزيز سلامة البيانات ؛ حيث تتيح إجبار المتغير على اتخاذ نمط بيانات محدد—مثل تحويل سلسلة نصية رقمية إلى عدد صحيح—مما يضمن دقة العمليات، ويحول دون وقوع أخطاء برمجية ناتجة عن عدم توافق أنواع البيانات أثناء معالجة مدخلات النظام.

```
$grade = "95"; // قيمة نصية
settype($grade, "integer"); // التحويل إلى عدد صحيح
```

دالة فحص وجود المتغير (isset)

تُمثل آلية التحقق من وجود المتغير في ذاكرة النظام وتكمن قيمتها في توفير معايير أمنية للأنظمة التعليمية؛ حيث يتم توظيفها كأداة تدقيق قبلي للتأكد من أن المتغير قد تم تعريفه وإسناد قيمة له قبل استدعائه، مما يحمي التطبيق من التوقف المفاجئ عند معالجة متغيرات غير مُعرفة أو فارغة.

```
$student_id = "12345";
if (isset($student_id)) {
    echo "المتغير موجود ومسجل في النظام";
}
```

دالة فحص فراغ الحقول empty()

الدالة المخصصة للتحقق من خلو المتغيرات من القيم وتؤدي وظيفة جوهرية في واجهات التفاعل؛ إذ تُستخدم للتحقق من اكتمال مدخلات الطالب (مثل الإجابات أو البيانات الشخصية)، مما يضمن التزام المتعلم بتعبئة كافة الحقول المطلوبة قبل إتمام عملية الإرسال، وهو ما يعزز من كفاءة التفاعل داخل النظام.

```
$answer = ""; // حقل فارغ
if (empty($answer)) {
    echo "عذراً، يرجى تعبئة حقل الإجابة";
}
```

دالة حذف المتغير unset ()

تُعد هذه الدالة بمثابة أداة إدارة موارد الذاكرة؛ حيث تقوم بإلغاء تخصيص المتغير ومسحه نهائياً من الذاكرة. وتتجلى أهميتها العملية في الحفاظ على كفاءة النظام وأمن البيانات؛ إذ يتم توظيفها لحذف البيانات المؤقتة فور انتهاء الغرض منها، مما يضمن حماية خصوصية المتعلمين وتحرير موارد السيرفر، وهو أمر حيوي في الأنظمة الأكاديمية متعددة المستخدمين.

```
$temp_session = "بيانات مؤقتة";
unset($temp_session); // حذف المتغير من الذاكرة
echo "تم مسح البيانات المؤقتة بنجاح";
```